Tutorial 9: Object Detection using SSD & Yolo (Custom Model)

Name: Laiba Niazi

Roll Number: 536672

1. Introduction

This report summarizes the implementation and evaluation of two prominent object detection architectures—Single Shot Multibox Detector (SSD) and You Only Look Once (YOLO)—as detailed in **Tutorial 9**. The tutorial focuses on building custom backbones, integrating a custom raccoon dataset, and utilizing industry-standard pre-trained models for real-time detection.

2. Dataset Integration

A custom dataset class, RaccoonDataset, was implemented to handle raccoon imagery.

- **Preprocessing**: Images are resized to a target size (e.g., 300×300 for SSD) and converted to normalized tensors.
- **Labeling**: Due to the absence of local XML annotation files in the tutorial environment, a simple class label (Class 1) was assigned to represent the "Raccoon" category for initial testing.

3. Single Shot Multibox Detector (SSD) Implementation

The SSD model was constructed using a modular approach consisting of a backbone and a prediction head.

- **Backbone Network**: A custom CNN was defined to extract feature maps, utilizing multiple convolutional layers followed by ReLU activation and max-pooling.
- **Prediction Head**: Separate classification (cls_head) and bounding box (box_head) layers were used to predict probabilities and coordinates (x, y, w, h) respectively.
- **Multi-Scale Feature Maps**: The model predicts across three different scales by downsampling the feature maps, allowing it to detect objects of varying sizes.

```python
# Step 4: Dummy data testing for SSD
ssd_input = torch.randn(1, 3, 300, 300)
ssd_cls, ssd_box = ssd_model(ssd_input)
print(f"SSD Class Prediction Shape: {ssd_cls.shape}")
print(f"SSD Box Prediction Shape: {ssd_box.shape}")
```

[18] ✓ 0.0s Python

```
SSD Class Prediction Shape: torch.Size([1, 44580, 2])
SSD Box Prediction Shape: torch.Size([1, 44580, 4])
```

4. You Only Look Once (YOLO) Implementation

The tutorial implemented a version of the YOLO architecture, which divides images into a grid for detection.

- **Sequential Backbone**: A 5-block sequential CNN extracts features from a 416x416 input image.
- **Grid Output**: The YOLO head reshapes the output into a 13×13 grid, where each cell predicts bounding boxes and confidence scores.

```python
yolo_input = torch.randn(1, 3, 416, 416)
yolo_pred = yolo_model(yolo_input)
print(f"YOLO Prediction Shape: {yolo_pred.shape}")
```

[22] ✓ 0.0s Python

```
YOLO Prediction Shape: torch.Size([1, 13, 13, 3, 6])
```

5. Task 1: Improved Backbone with Batch Normalization

To enhance model stability and training speed, an ImprovedBackbone was developed for the SSD model.

- **Modification**: Batch Normalization (nn.BatchNorm2d) was added after each convolutional layer.
- **Verification**: The improved model was tested on a real raccoon image, successfully generating predictions for 44,580 anchor boxes.

```
--- Task 1 Output ---
Processed 1 Raccoon image of shape: torch.Size([1, 3, 300, 300])
SSD Improvements (Batch Norm) verified. Predictions generated for 44580 anchor boxes.
```

6. Task 2: Pre-trained Model Evaluation

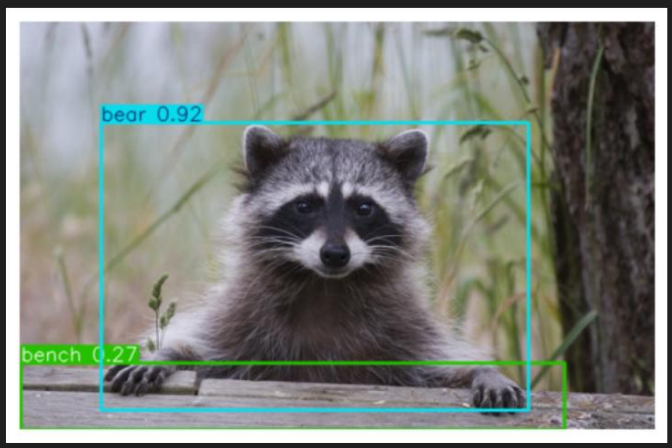
The tutorial compared the custom implementations with high-performance pre-trained models:

1. **SSD300 VGG16**: A standard SSD model pre-trained on the COCO dataset.
2. **YOLOv8 Nano**: The lightweight version of the Ultralytics YOLOv8 model.

Execution Results:

Testing the pre-trained YOLOv8 model on a sample raccoon image resulted in the detection of multiple objects, providing exact bounding box coordinates in a tensor format.

```
--- Task 2 Output ---
Detected 2 objects in the image.
Bounding Box Coordinates:
tensor([[ 83.9871, 104.5142, 520.0626, 397.8418],
        [  1.4193, 349.5530, 557.5766, 417.0000]])
```



7. Conclusion

The tutorial successfully demonstrates the transition from building basic object detection backbones to utilizing state-of-the-art pre-trained models. While custom models provide insight into architecture design, pre-trained versions like YOLOv8 offer immediate, high-accuracy results for complex datasets.